

RoboReason: A Real-Hardware Harness for Code-Grounded LLM/VLM Manipulation

Matteo Nini¹

Abstract—Vision-language-action (VLA) models fold perception, reasoning, and control into one policy, but need paired vision-action data far scarcer than the text and image corpora an LLM or VLM is pretrained on. We argue instead for a decoupled Sense-Plan-Act architecture: an LLM/VLM plans in language, and a deterministic code layer grounds the plan’s geometry in sensor measurement, so pretrained knowledge transfers through code without task-specific action data. We present a real-hardware ROS2 harness built on this principle. Deploying it on a UR5cb manipulator surfaced six recurring geometry-grounding bugs invisible to symbolic simulation by construction; the code-grounding layer closes five, and a 120-trial benchmark shows the sixth, a multi-object placement collision, as the dominant residual failure in both an LLM and a camera-grounded VLM arm. We report this taxonomy as a discovered validation of the harness’s premise, not our central claim.

I. INTRODUCTION

A vision-language-action (VLA) model maps perception and instruction directly to low-level control, in one policy trained end-to-end. This is an elegant architecture, but it inherits imitation learning’s oldest bottleneck: the policy only knows what its demonstration data covers, and paired vision-action robot trajectories are expensive to collect at the scale a general-purpose manipulator needs. An LLM or VLM, by contrast, is pretrained on a far broader, cheaper-to-collect corpus, text, code, and image-text pairs at internet scale, and already knows a great deal relevant to manipulation, object affordances, spatial and arithmetic reasoning, ordering and precondition logic, none of it robot-specific, all of it transferable through code rather than a demonstration dataset [1]–[4]. This is the architectural bet a decoupled Sense-Plan-Act pipeline makes: let the model plan in language, and a separate execution layer turn that plan into motion.

The bet has a cost. A decoupled pipeline has to ask the model, somewhere, to commit to numbers, a release height, a grasp width, a target position, and nothing in an LLM or VLM’s pretraining grounds those numbers in the physical world it now acts in. Simulated validation hides this cost almost entirely: an object’s height, its grasp width, and the distance a gripper must travel to release it safely are never actually *measured* there, they are asserted, correctly, by construction [5]–[7]. On real hardware none of these quantities are given; they have to be computed, from noisy depth, an imperfect camera, or a hand-authored scene file that may not match the physical table it describes.

We present a real-hardware ROS2 harness built around this architectural choice (§III): a modular Sense-Plan-Act pipeline supporting six reasoning strategies over two grounding modalities (a static-scene LLM planner and a camera-grounded VLM planner), a small library of parameterized manipulation skills, and a deterministic code-grounding layer that overrides model-computed geometry with a sensor- or action-derived value wherever the two could disagree. A companion framework from the same group, RoboReason Lab [8], implements the same class of reasoning strategies over the same kind of pipeline, but entirely inside CoppeliaSim; ours, as far as we are aware, is the first version of it run on a real manipulator.

Building and operating this harness on a real UR5cb manipulator surfaced the cost concretely. Over six development sessions we found a recurring pattern: whenever a planning prompt asked the model to compute a piece of manipulation geometry itself, that computation was often wrong, in ways that stayed silent until the arm dropped, crushed, or missed an object. We diagnose six such bugs from production hardware logs (§IV) and close five with the harness’s code-grounding layer, which computes the same quantities deterministically instead of trusting the model’s own arithmetic (§V). The sixth, a multi-object placement collision, we diagnose but have not closed; it turns out to be the dominant one, a 120-trial benchmark (§VI) shows it as the residual failure once the other five are fixed, in *both* the LLM and VLM pipelines regardless of which grounded the scene. We read this taxonomy as a discovered validation of the harness’s design premise, not this paper’s main contribution: an LLM/VLM harness needs code-grounding precisely because real deployment keeps asking the model to compute quantities only a sensor, or an already-committed action, should determine.

The contributions of this paper are threefold: nousep

- A real-hardware ROS2 harness for decoupled LLM/VLM manipulation planning, with a code-grounding layer as an explicit architectural component, presented as the real-hardware counterpart to simulation-only frameworks for the same class of reasoning strategies (§III).
- A taxonomy of six recurring geometry-grounding bugs, discovered while operating the harness on a real manipulator across six development sessions, five of which the code-grounding layer closes (§IV, §V).
- A 120-trial real-hardware benchmark, methodologically anchored to a directly comparable symbolic-simulation study [5] and using a single unified model for both

¹The authors are with [Department, University]. E-mail: matteo.nini@[domain].

grounding modalities, showing that the sixth, still-open bug, multi-object placement collision, is the dominant residual failure in *both* arms, and what that implies for evaluating LLM/VLM robot harnesses before hardware deployment (§VI).

II. RELATED WORK

From symbolic task-and-motion planning to LLM-based planning. Robotic planning has long been structured around a Sense-Plan-Act pipeline [9], originally realized through hierarchical and symbolic planners that search a discrete action space for a sequence provably achieving a goal condition [10]. These planners give strong correctness guarantees but require a hand-specified symbolic model of the world and its dynamics. Foundation models have been used to ground natural-language instructions in robotic affordances [11], to act as zero-shot planners [12], and to generate feasible motion plans directly from instructions [13]. A range of reasoning strategies has been proposed on top of these capabilities: reactive, single-step prompting; interleaved reasoning-and-acting (ReAct) [14]; iterative self-critique (Self-Refine) [15], including a long-horizon sequential-planning variant validated at ICRA [16]; search-based deliberation (Tree-of-Thoughts) [17]; and ensemble voting over independently sampled plans (chain-of-thought with self-consistency) [18], [19]. Recent surveys catalogue this space in detail [20]–[24]. Our contribution is orthogonal to this line of work: we do not propose a new reasoning strategy, and hold the reasoning strategy fixed in our evaluation (§VI). Instead, we show that the geometric grounding surrounding *any* of these strategies, how a chosen action’s numeric parameters are computed, is a separate failure axis that persists regardless of which reasoning strategy produced the action.

End-to-end and vision-language-action approaches. An alternative architecture collapses perception, reasoning, and control into a single model, as in vision-language-action (VLA) systems [25]–[28]. Trained or fine-tuned end-to-end, such a policy can in principle learn to compensate for geometric grounding errors on its own. It pays for that with the one resource imitation learning has always been bottlenecked on: paired vision-action demonstration data, collected per embodiment, which stays scarce and narrow next to the text/code/image-text corpora an LLM or VLM is pretrained on. Code-generating policies that couple perception directly to control primitives [1]–[4] sit closer to our own approach: they keep the decoupled Sense-Plan-Act split and let an LLM’s pretrained code and spatial-reasoning knowledge stand in for embodiment-specific demonstrations. A fully end-to-end VLA also gives up the diagnosability a decoupled pipeline provides: when it fails, it is difficult to determine whether the failure was a misperception, a reasoning error, or a motor-control error [5]. We adopt the decoupled architecture for both reasons at once, data efficiency and diagnosability, since a fully end-to-end system would not permit the diagnosis of §IV at all, only its symptoms.

Simulation and sim-to-real transfer. The sim-to-real gap is a long-studied concern in robot learning, primarily for learned low-level control policies [29]. Physics simulators such as Gazebo [30], MuJoCo [7], and CoppeliaSim [6] narrow this gap for control by modeling contact dynamics and actuation limits [31]. The gap we document is different in kind. It does not concern whether a *learned policy* generalizes from simulated to real dynamics, but whether a *language-model-computed* geometric quantity, one that a symbolic benchmark never forces the model to compute against a contradicting sensor, is trustworthy at all. To our knowledge this failure class has not been previously isolated and named for LLM/VLM robot planners specifically.

Direct comparison. The closest prior work to this paper is Favali et al. [5], which introduces a task/environment complexity taxonomy and TS/TSR/AETS metrics for benchmarking LLM-based robotic planning across six reasoning strategies, evaluated in a symbolic simulator with exact ground truth. We adopt the same evaluation philosophy and, for §VI, the same metric family, but on real hardware, where ground truth for safety and success is not automatically available (§VI details the resulting semi-automatic adaptation). Neither it nor its companion systems paper, RoboReason Lab [8] (§III-B), ever grounds a plan in measured depth or a calibrated camera; both are simulation-only by design, and so structurally unable to produce the failures §IV diagnoses.

III. THE HARNESS

A. Design rationale: code-grounding over end-to-end control

A vision-language-action (VLA) model collapses perception, reasoning, and control into one policy, trained or fine-tuned end-to-end on embodiment-specific demonstration data, which is the bottleneck: paired vision-action trajectories remain far scarcer and more expensive to collect than the text, code, and image-text corpora a general-purpose LLM or VLM is pretrained on. A decoupled Sense-Plan-Act architecture sidesteps this. If planning is text (or text conditioned on an image), and the resulting plan is a sequence of parameterized skills rather than raw joint torques, then whatever manipulation-relevant knowledge the model already has, spatial reasoning, arithmetic, common-sense physics, ordering and precondition logic, is available zero-shot, through code, without a single task-specific demonstration, the same argument prior code-generation-for-robotics work makes [1]–[4]: an LLM that has never seen a robot trajectory has still seen a great deal of code.

The cost is that a decoupled architecture asks the model to compute physical quantities, a release height, a grasp width, a target position, that it did not learn from a source grounded in the real world at all; §IV is what happens when that computation is trusted uncritically. Our answer is not to abandon the decoupled architecture but to constrain it: let the model choose *what* to do and *where*, symbolically, and let a deterministic layer compute *how much*, from a sensor measurement or a paired action already in the plan, whenever

the plan needs a number the model cannot be trusted to produce on its own.

B. System architecture

The harness is a ROS2 stack organized around the same Sense-Plan-Act pipeline as §II, shown in Figure 1. A task interface (a terminal client or a web GUI) forwards a natural-language command to one of three planner nodes: a static-scene LLM planner, a camera-grounded VLM planner, or a hybrid that runs a VLM scene-grounding call first and hands the resulting scene to the same LLM planner used in static-scene mode. Whichever planner produced it, the plan is validated and sequenced by a plan-manager node, which drives a skill-executor node (a physics-free mock for fast iteration, or the real UR5cb driver) through a small, fixed set of parameterized skills: `approach`, `pick`, `release`, `move_home`, and `wait`. Every planning call supports six interchangeable reasoning strategies, from a single-step reactive baseline to self-consistency voting and tree search, selectable without touching the pipeline itself.

Two design choices let this pipeline scale into a research tool rather than a one-off demo. First, every runtime parameter, model choice, reasoning strategy, grounding modality, timeouts, robot IP, is a field on one centralized settings object, overridable by an environment variable with no code change, which is what lets §VI’s conditions be defined and reproduced as configuration rather than as code branches. Second, every planning call records a per-run debug artifact, the command, active configuration, raw model response, execution log, and, in VLM modes, the camera frame the plan was grounded in, which is what made diagnosing §IV’s failures possible: each bug was traced from a hardware failure back to a specific malformed field in a specific logged response, not inferred after the fact. A web GUI supervises the whole stack, including the physical UR driver and camera service, both of which fail intermittently enough to need their own restart logic, as monitored child processes, so a trial can be launched, watched, and annotated without a terminal per node.

IV. GEOMETRY-GROUNDING FAILURES DISCOVERED ON REAL HARDWARE

Operating the harness of §III on real hardware turned the design rationale of §III-A from an argument into an observation: six recurring geometry-grounding bugs, each traced to its root cause using the harness’s own per-run debug artifacts (raw model response, config, and, in VLM mode, the captured camera frame). Table I summarizes all six. Below, we develop the two most instructive of the five closed cases in full, since both recurred independently in two different planning pipelines and so best show that the pattern is structural, not a one-off slip, then treat the sixth, still-open bug on its own (§IV-C), which turns out to be the dominant failure mode in §VI.

A. Case 1: top-vs-mid-body grasp/release mismatch

Our executor deliberately grasps objects mid-body rather than at their top surface: the pick descent is $d =$

$\min(0.5h, d_{\max})$, where h is the object’s height and $d_{\max}$ a clamp preventing the gripper’s rigid mounting body from crashing into tall objects, since centering the grasp mid-body produces a more stable hold than pinching at the top edge.

The paired *release* action, however, computes how far to lift the object before opening the gripper (`object_height`, added to `release_position.z` at execution time) by assuming it was grasped at the top, wrong by construction whenever the pick descended mid-body, which is always. The result is a systematic overshoot of roughly $0.5h$: objects land, or are dropped from, a height proportional to half their own size rather than fractions of a centimeter.

We fixed this once, in the VLM pipeline, by deriving `object_height` from measured depth ($h = \max(z_{\text{surf}} - z_{\text{top}}, h_{\min})$) instead of a model-reported size. It then recurred independently in the LLM pipeline weeks later, because that pipeline’s prompt still asked the model to compute `object_height` as `size[2]`, the same assumption the VLM-mode fix had already identified as wrong. Fixing the bug once, in one pipeline, did not fix it everywhere it existed; the general fix (§V) computes this quantity deterministically for *every* pipeline, since the error is not a mistake any particular prompt makes but a mismatch between what the model is asked to assume and what the gripper actually does.

B. Case 2: double-counted stacking height

Our static scene description uses the convention `position.z` = base reference, `size[2]` = height, so `position.z + size[2]` reconstructs a target’s top surface, the formula every LLM-mode prompt uses for a stacking release height. In our VLM→LLM hybrid pipeline, a VLM grounding call first deprojects a target’s pixel location to a depth-measured world coordinate, then hands the resulting scene to the same LLM planner used in static-scene mode. An early version of this hybrid set the generated scene’s `target.position.z` directly to that depth-measured top surface, already correct, while leaving the VLM’s separate, un-verified `size[2]` guess untouched. The downstream planner then applied its normal stacking formula, adding the guessed height on top of an already-correct value and inflating every stack-on-target release by however wrong that guess happened to be.

The fix follows Case 1’s pattern: derive the target’s real height from depth ($h = \max(z_{\text{top}} - z_{\text{surf}}, 0)$, no minimum floor, since a flat target zone marked on the table should read as near-zero height), then reset `position.z` to a base reference so the existing downstream formula reconstructs the true top surface instead of double-counting it. The sensor-computed quantity is inserted *into* the representation the model already expects, rather than trusted from the model’s own output.

C. Case 6 (open): multi-object sequential-placement collision

Unlike Cases 1–2, this bug is not closed by §V. When a task places several objects into the same target zone in sequence, each release’s *final* position is spaced to avoid

System architecture diagram: task interface/GUI, the three planner nodes, the plan manager, the skill executor, and the code-grounding layer, with the debug-artifact and configuration paths annotated.

(placeholder; final figure to be supplied)

Fig. 1. RoboReason harness architecture: the Sense-Plan-Act pipeline of §III-B, with the code-grounding layer overriding model-computed geometry before execution.

TABLE I

GEOMETRY-GROUNDING FAILURE TAXONOMY. “SENSOR/ACTUATOR REQUIRED” MARKS THE PHYSICAL PROPERTY THAT MAKES EACH BUG IMPOSSIBLE TO OBSERVE IN A SYMBOLIC OR SCRIPTED SIMULATOR.

Bug	Symptom and root cause	Sensor/actuator required
Top-vs-mid-body grasp/release mismatch	Release-lift height set to an object’s full height, implicitly assuming a top-surface grasp; the executor actually descends mid-body ($d = \min(0.5h, d_{\max})$). Net effect: a systematic release overshoot of roughly half the object’s height. Recurred independently in two pipelines (VLM mode, then LLM mode).	Real gripper kinematics (mid-body descent is a deliberate stability choice, not a simulator default).
Double-counted stacking height	In a VLM→LLM hybrid pipeline, a depth-measured true top surface had the VLM’s separate, non-depth-verified size estimate added on top of it a second time by the downstream LLM’s own stacking formula ($\text{position.z} + \text{size}[2]$).	Real depth measurement of an already-correct surface height.
Footprint- vs. point-distance target matching	A release genuinely inside a tray’s real bounding box, but far from its registered center point, was treated as having missed the target zone by a point-distance check.	Real target geometry with non-trivial spatial extent (tens of centimeters), not a point.
Grasp-width sampled from the silhouette edge	Grasp width derived by deprojecting a bounding box’s left/right edge pixels, exactly on the object’s silhouette boundary, the single worst location for valid depth (occlusion/parallax). One failed edge sample aborted an otherwise valid plan.	Real depth noise at object boundaries; does not exist for a rendered or symbolic bounding box.
Width-dependent TCP offset	The gripper’s fingers pivot, so flange-to-contact distance is a function of how far apart they are commanded to open, not a constant, so it is wrong for any object whose width diverges from a single calibrated offset.	Real gripper mechanism (finger kinematics), absent from a symbolic action space.
Multi-object sequential-placement collision (open)	Zone-aware spacing keeps a target zone’s <i>final</i> release positions from overlapping, but does not prevent the arm clipping an already-placed object en route to the next release, nor a released object settling into a neighbor from residual momentum or a few cm of height error. Dominant failure mode in a 120-trial re-evaluation: every non-empty operator note (22/22) describes a collision or a fallen/displaced object during a multi-object task, in both the LLM and VLM pipelines.	Real multi-body contact between two already-placed objects and the arm’s approach trajectory, absent from any single-object placement check, symbolic or real.

overlapping the others already placed, by a grid clamped to the zone’s real bounds (§V). This closes the *static* half: no two final positions occupy the same footprint. It does not address the *dynamic* half: nothing constrains the arm’s trajectory while approaching or releasing the n -th object against the $(n-1)$ -th already sitting in the zone, and nothing constrains a released object’s small residual motion from displacing a neighbor afterward. Both are genuinely three-dimensional, time-dependent contact problems, not resolvable by correcting a single scalar the way Cases 1–2 were.

We identified this bug from the same debug artifacts as Cases 1–2, but by the time it was isolated it was already the dominant failure mode of §VI (22 of 22 non-empty operator notes, both pipelines), rather than something we could close before reporting results. We report it as an

identified, partially-mitigated **open** problem, not a claimed contribution; §VII returns to what closing it would require.

D. The common pattern

The six bugs in Table I share a structure: a quantity trivially, exactly correct in a symbolic or scripted simulator becomes, on real hardware, (a) a sensor measurement subject to depth noise and occlusion, (b) a function of real actuator kinematics a text-based action schema has no natural way to express, or (c), for Case 6, a consequence of real multi-body contact over time. None of these three can occur where object geometry is authored by the experiment designer or perfectly known by the physics engine, precisely the setting where LLM/VLM robot planners are most commonly benchmarked [5], [8].

V. CLOSING THE DISCOVERED FAILURES

A. Design principle

The fix for five of the six bugs in Table I follows the same rule: *pull geometric arithmetic out of the model and into code wherever the answer is computable from a sensor measurement or a paired action already in the plan, leaving the model responsible only for the symbolic decision of what to do and where, not how much*. The model still chooses which object to pick and which zone to release into; a deterministic post-processing layer overwrites the specific numeric fields most exposed to §IV’s failure modes, using only information already available at plan time. The sixth bug (Case 6, §IV-C) is not a scalar this rule can overwrite, since it is a trajectory-level, time-dependent contact problem, and is addressed separately, and only partially, below.

B. Object and release height

Given a generated plan, `fix_object_height` recomputes every release step’s `object_height` as the true vertical distance from the paired pick’s actual grasp contact point to the table surface, $h-d$ in the notation of §IV-A, rather than accepting whatever height value the model reported. This directly closes Case 1: the release-lift amount is now derived from the *same* descent the executor actually commanded during the pick, which by construction cannot disagree with itself, instead of from a separate, independently-computed model output that could.

`fix_release_height` performs the analogous correction for the release *position*. Matching a release against the scene’s registered targets is done by **footprint containment** (position $\pm$ size/2) rather than point-distance to a target’s center, since Case 3 showed point-distance matching misses valid placements inside a target zone tens of centimeters wide. A release position that *exactly echoes* a target’s or object’s own registered coordinates is treated as intentional and left untouched, while one that merely *drifted* into a zone’s footprint is nudged back outside along whichever axis needs the smaller correction, and only when that zone’s surface sits meaningfully above the bare table, since nudging away from the near-table-height generic “table” zone would otherwise push objects off the table’s edge escaping a zone that legitimately contains most of the workspace.

C. Width-aware TCP offset

Because the gripper’s fingers pivot, the flange-to-contact distance for a given commanded width is not constant. We measure this at three representative widths (fully closed, mid-open, fully open) and interpolate piecewise-linearly between them at plan time, setting the executor’s tool transform per-pick from the plan’s reported grasp width and resetting it to a default afterward. Pick descent is additionally capped at (calibrated offset – clamp clearance), so tall objects are gripped nearer their top rather than driving the gripper’s rigid mounting body into them. This closes Case 5.

D. Zone-aware spacing: a partial mitigation for Case 6

For a target zone that receives multiple releases in one plan, we lay out occupants on a grid centered on the zone’s anchor and clamped to its registered bounds, falling back to an unbounded grid only when no known zone matches. This closes the *static* half of Case 6: final release positions no longer overlap or overflow a zone smaller than the model’s own unclamped guess would have. It does not close the *dynamic* half: the arm’s approach trajectory to the n -th release is not checked against the already-placed $(n-1)$ -th object’s footprint, nor is a released object’s small residual motion constrained at all. §VI reports this as still the dominant failure mode; §VII returns to what closing it would require.

E. Best-effort grasp-width sampling

For the VLM pipeline, grasp width is estimated by deprojecting a detected bounding box’s left and right edge pixels. Sampling directly on the object’s silhouette boundary is the single worst location for valid depth, since boundary pixels are the most exposed to occlusion and parallax error, and a single failed sample previously aborted the entire batched deprojection call over a parameter, grasp width, that does not even affect whether *pick* and *release* themselves are safe. We fixed this by sampling slightly inset from the raw silhouette edge instead of on it, and by isolating width estimation into its own best-effort call that degrades to the model’s own guess on failure instead of aborting the plan. This closes Case 4 without requiring width to be depth-verified in every case, an open limitation we return to in §VII.

F. Validation

Each fix in this section was developed and iterated against real, previously-failing production runs: the per-run debug artifacts our system records for every planning call (raw model response, active configuration, and, in VLM modes, the captured camera frame) let us replay the exact inputs that produced a given hardware failure and confirm that the corrected pipeline now produces the hand-derived expected geometry for that same input, rather than validating only against synthetic test cases. This retrospective replay, together with a unit-level regression suite (45/45 passing across all reasoning-method implementations at the time of writing), gives us confidence that each fix addresses its diagnosed root cause. §VI reports the residual failure distribution with the harness’s code-grounding layer in place.

VI. REAL-HARDWARE EVALUATION

A. Design

With the deterministic grounding layer of §V in place, we ran a 120-trial real-hardware study comparing a static-scene LLM planner against a camera-grounded VLM planner, anchored to the task/environment complexity taxonomy and TS/TSR/AETS metrics of Favali et al. [5], adapted to a physical rig. Six task conditions crossed three difficulty axes, task length, reference specificity, and affordance/perception

TABLE II

OVERALL RESULTS, 60 TRIALS PER MODEL (120 TOTAL), WITH THE DETERMINISTIC GROUNDING LAYER OF §V IN PLACE.

Model	n	TS	TSR	AETS
LLM (kimi-k2.6)	60	98.3%	98.3%	0.1269
VLM (kimi-k2.6)	60	85.0%	89.2%	0.1149

load, each easy and hard, over a real scene of four color-distinguished cubes and a tray: *pp_easy/pp_hard* (pick-and-place, one vs. four objects), *sort_easy/sort_hard* (stacking and sorting), and *arith_easy/arith_hard* (the object must be *computed* from a stated numeric mapping rather than named, isolating reasoning load from perception load). Each condition ran 10 times per model, 120 trials total, with a single reasoning method (chain-of-thought with self-consistency) held fixed, the most safety-robust strategy per [5] and one that avoids stacking reasoning-method variance on top of the comparison under test. Both planners use the same underlying model (kimi-k2.6), text-only and image-grounded respectively, so grounding modality is the only variable isolated between arms.¹

B. Metrics, adapted for physical ground truth

Favali et al.’s TS (task safety), TSR (task success rate), and AETS (action efficiency on task success) are fully automatic in a symbolic simulator, where whether an action “damages the environment” or a sub-task “completes” is always exactly derivable from simulator state. Our real UR5cb has no collision or force-fault sensor anywhere in its control path and no vision-based verification of the final scene, so we adapt the same three metrics to a **semi-automatic** protocol: everything the system can know automatically is logged by the debug harness of §V for every trial, and the operator supplies exactly two numbers immediately after, was it safe and how many sub-tasks completed, from which TS, TSR, and AETS are computed identically to [5]’s definitions.

At $n = 10$ per cell, point estimates alone invite exactly the kind of pushback a reviewer should raise, so every reported TS% and TSR% below carries a 95% Wilson score interval (TS is an exact per-trial proportion; TSR pools successes over $\text{sub_tasks_required} \times n$). One comparison was declared before any data was collected, not chosen afterward to fit the numbers: a two-sided Fisher’s exact test, LLM vs. VLM, on *arith_hard* full-trial success, at $\alpha = 0.05$. It is binarized per trial rather than pooled across a trial’s four sub-tasks, since those are not independent draws; one wrong decision tends to misplace every object in the trial together, not each independently. Everything else is reported descriptively only.

C. Results

Table II reports the headline result. Both models share the same grounding layer and the same underlying model, so the

¹An earlier run used different models per modality; the VLM-specific one was later withdrawn from its provider’s catalog, forcing a full re-run on a single unified model instead.

Overall headline results — LLM vs VLM (n=60 per bar, 95% Wilson CI)

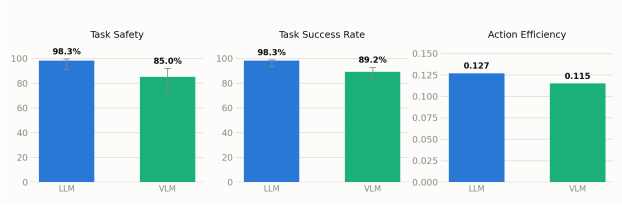


Fig. 2. Overall TS/TSR/AETS summary, LLM vs. VLM, with the deterministic grounding layer in place.

Safety and success rate by task — LLM vs VLM (n=10 per bar, 95% Wilson CI)

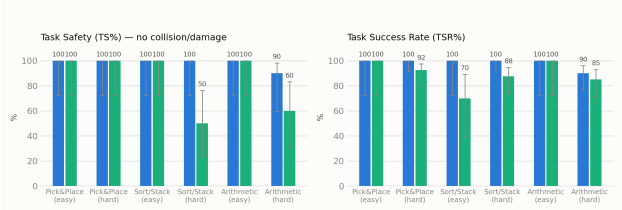


Fig. 3. Task safety and task success rate by task condition, with 95% Wilson CI.

residual gap is neither a grasp-geometry gap nor a model-capability gap; it is attributable to grounding modality. By task condition (Figure 3) the gap concentrates on the two hardest multi-object conditions, and predominantly in *safety*: TS is 100% for both models on every easy condition and on *pp_hard/sort_hard* (LLM), but drops to 50% (VLM, *sort_hard*) and 60%/90% (VLM/LLM, *arith_hard*) where the most objects share a zone at once. Neither model shows the sharp *success-rate* collapse a July pilot with different, modality-specific models found on the arithmetic conditions (30%/25% VLM TSR against 100%/97% LLM): with a single unified model, *arith_easy* is 100%/100% and *arith_hard* is 90%/85% (TSR, LLM/VLM), a real but modest gap, not a collapse (§VI-D). The one pre-registered test, Fisher’s exact, LLM vs. VLM on *arith_hard* full-trial success, shows no detectable difference at this sample size (LLM 6/10, VLM 4/10, $p = 0.66$), consistent with a real but modest gap $n = 10$ per cell cannot resolve to significance.

D. The failure mode is computation, not perception

Reading all 22 non-empty operator failure notes from the 120 trials, rather than trusting aggregates alone, was the key analysis step. Every note describes either a collision between objects (10) or an execution/grasp failure, a slipped grasp, a release too high, a dropped object (12), during a multi-object task. **Zero** describe a wrong object chosen for reasoning or value-mapping reasons, and zero an incomplete enumeration (Figure 4). This happens in **both** arms: the LLM contributes 4 of the 22 notes (all on *arith_hard*), the VLM the remaining 18, spread across all three multi-object conditions roughly in proportion to trial counts. The failure recurs regardless of which planner grounded the scene, direct confirmation that Case 6 (§IV-C) is this taxonomy’s most consequential open item, not a peculiarity of one pipeline.

Unlike the closed bugs of §IV, this result is not a second-

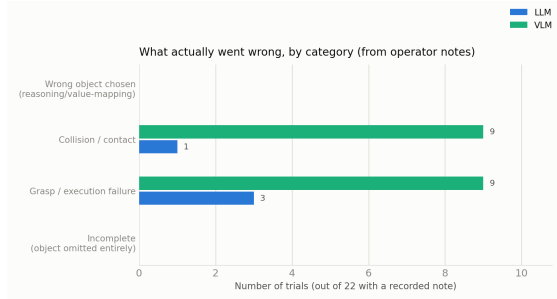


Fig. 4. Failure-mode breakdown, derived from the 22 non-empty operator notes out of 120 trials.

order instance of the paper’s central claim; it is a direct one. Case 6 is itself one of the six geometry-grounding bugs of Table I; this evaluation is what identifies it as the dominant one once the other five are closed.

A note on an earlier finding. A pilot on this same task matrix, using different, modality-specific models, found a different dominant category instead: 13 of 31 notes read as a wrong-object choice concentrated in the arithmetic condition. That finding does not replicate here, zero such notes this cycle, on every condition, and we read the simplest explanation as the correct one: a property of the earlier VLM model specifically, not of VLM grounding in general, and a caution against generalizing a single-model finding to a category.

VII. DISCUSSION AND LIMITATIONS

Implication for evaluation practice. Symbolic and scripted simulation is necessary but not sufficient for validating an LLM/VLM robot planner: it validates reasoning strategy and task decomposition, but by construction cannot exercise the geometry-grounding failure class of §IV, since it never asks the model to compute a quantity a physical sensor could contradict. A geometry-grounding audit, identifying every numeric quantity a prompt asks the model to compute itself and asking whether a sensor or paired action already determines it more reliably, belongs alongside task-complexity benchmarking as a standard step before hardware deployment.

The cost of the LLM/VLM-vs-VLA choice. §III-A argued for code-grounding over an end-to-end VLA on data-efficiency grounds. §IV is the other half of that trade: six bugs is the price of asking a model to compute physical quantities it was never trained to get right, and §V is what it costs to pay that price down. We read this as a cost worth paying, not a case against the architecture, since every bug in Table I was closeable once diagnosed, precisely because the pipeline stayed decoupled enough to diagnose it.

Case 6 remains the largest open item. Everything else in this paper is either closed or diagnosed precisely enough to know what closing it would take. Case 6 is neither: it is a trajectory-level, multi-body contact problem, and §V-D’s static-spacing mitigation does not touch its dynamic half. Closing it plausibly needs a motion-planning-level check of the approach trajectory against already-placed objects’

known footprints, or a settle-time verification step after each release so a displaced neighbor is caught before the next pick. We have not implemented or evaluated either, but the first is naturally an obstacle-avoiding approach skill on the harness of §III, treating already-placed objects the same way a mobile-base planner already treats static obstacles, and is the concrete next addition we have in mind.

Generalizability. Our evidence comes from one manipulator (UR5cb), one gripper, one camera rig, and a single underlying model (`kimi-k2.6`) used for both the LLM and VLM arm. We do not claim that the specific six bugs of Table I recur verbatim on different hardware, only that the *structural pattern*, geometry computed by a model instead of measured, and multiple real bodies sharing space over time, is a property of this architecture, not of this robot. Grasp width is the one quantity we still cannot cross-check against depth in every case, since a single top-down sample determines height unambiguously but not left-right extent; our best-effort edge-inset sampling (§V) reduces this gap without closing it.

Reliability of operator-supplied labels. Our semi-automatic protocol (§VI) rests on a single operator’s real-time judgment, unverified by a second, blinded annotator. This is not a hypothetical risk: an early pass over this cycle’s data marked every trial safe regardless of outcome, caught only by cross-checking the safety label against the operator’s own free-text note, and corrected before §VI’s numbers were computed.

VIII. CONCLUSION

We presented a real-hardware ROS2 harness for decoupled LLM/VLM manipulation planning, built on the premise that a model’s broad, non-robot-specific pretraining is usable for physical reasoning through code, without collecting task-specific action data, provided a deterministic layer grounds the plan’s numeric geometry in sensor measurement rather than trusting the model’s own arithmetic. Deploying that harness on a real UR5cb manipulator turned this premise into an observation: over six development sessions we found six recurring geometry-grounding bugs, invisible to symbolic and scripted simulation by construction, and closed five of them with the harness’s code-grounding layer. A 120-trial real-hardware evaluation, using a single unified model for both grounding modalities, shows the sixth, a multi-object placement collision, as the dominant residual failure, occurring in both the LLM and VLM pipelines regardless of which one grounded the scene. Closing that bug’s dynamic, trajectory-level half as a new harness skill, along with a depth-verified treatment of grasp width, are the most direct next steps; more broadly, we argue that a geometry-grounding audit belongs alongside task-complexity benchmarking for any LLM/VLM harness headed to physical hardware.

REFERENCES

- [1] J. Liang, W. Huang, F. Xia, P. Xu, K. Hausman, B. Ichter, P. Florence, and A. Zeng, “Code as policies: Language model programs for em-

- bodied control,” in *2023 IEEE International Conference on Robotics and Automation (ICRA)*, pp. 9493–9500, IEEE, 2023.
- [2] W. Huang, C. Wang, R. Zhang, Y. Li, J. Wu, and L. Fei-Fei, “VoxPoser: Composable 3D value maps for robotic manipulation with language models,” *arXiv preprint arXiv:2307.05973*, 2023. Conference on Robot Learning (CoRL) 2023.
 - [3] W. Huang, F. Xia, T. Xiao, H. Chan, J. Liang, P. Florence, A. Zeng, J. Tompson, I. Mordatch, Y. Chebotar, P. Sermanet, N. Brown, T. Jackson, L. Luu, S. Levine, K. Hausman, and B. Ichter, “Inner monologue: Embodied reasoning through planning with language models,” *arXiv preprint arXiv:2207.05608*, 2022.
 - [4] I. Singh, V. Blukis, A. Mousavian, A. Goyal, D. Xu, J. Tremblay, D. Fox, J. Thomason, and A. Garg, “ProgPrompt: Generating situated robot task plans using large language models,” in *2023 IEEE International Conference on Robotics and Automation (ICRA)*, pp. 11523–11530, IEEE, 2023.
 - [5] F. Favali, L. Sabattini, and V. Villani, “LLM-based reasoning for robotic planning: Robustness to task and environmental complexity,” *Robotics and Autonomous Systems*, 2026. In press. TODO(authors): update volume/pages once assigned.
 - [6] E. Rohmer, S. P. N. Singh, and M. Freese, “CoppeliaSim (formerly V-REP): A versatile and scalable robot simulation framework,” in *Proc. of The International Conference on Intelligent Robots and Systems (IROS)*, 2013. www.coppeliarobotics.com.
 - [7] E. Todorov, T. Erez, and Y. Tassa, “MuJoCo: A physics engine for model-based control,” in *2012 IEEE/RSJ International Conference on Intelligent Robots and Systems*, pp. 5026–5033, IEEE, 2012.
 - [8] F. Favali, L. Sabattini, and V. Villani, “RoboReason Lab: An accessible simulation framework for foundation models in robotics.” Manuscript, Department of Sciences and Methods for Engineering (DISMI), University of Modena and Reggio Emilia. Code: <https://github.com/filippofavali/RoboReason-Lab>. TODO(authors): update with final venue/identifier once available., 2026.
 - [9] C. R. Garrett, R. Chitnis, R. Holladay, B. Kim, T. Silver, L. P. Kaelbling, and T. Lozano-Pérez, “Integrated task and motion planning,” *Annual Review of Control, Robotics, and Autonomous Systems*, vol. 4, pp. 265–293, 2021.
 - [10] L. P. Kaelbling and T. Lozano-Pérez, “Hierarchical task and motion planning in the now,” in *2011 IEEE International Conference on Robotics and Automation*, pp. 1470–1477, IEEE, 2011.
 - [11] M. Ahn, A. Brohan, N. Brown, Y. Chebotar, O. Cortes, B. David, C. Finn, C. Fu, K. Gopalakrishnan, K. Hausman, *et al.*, “Do as I can, not as I say: Grounding language in robotic affordances,” *arXiv preprint arXiv:2204.01691*, 2022.
 - [12] W. Huang, P. Abbeel, D. Pathak, and I. Mordatch, “Language models as zero-shot planners: Extracting actionable knowledge for embodied agents,” in *International Conference on Machine Learning*, pp. 9118–9147, PMLR, 2022.
 - [13] K. Lin, C. Agia, T. Migimatsu, M. Pavone, and J. Bohg, “Text2motion: From natural language instructions to feasible plans,” *Autonomous Robots*, vol. 47, no. 8, pp. 1345–1365, 2023.
 - [14] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafraan, K. R. Narasimhan, and Y. Cao, “ReAct: Synergizing reasoning and acting in language models,” in *The Eleventh International Conference on Learning Representations*, 2022.
 - [15] A. Madaan, N. Tandon, P. Gupta, S. Hallinan, L. Gao, S. Wiegrefe, U. Alon, N. Dziri, S. Prabhunoye, Y. Yang, *et al.*, “Self-refine: Iterative refinement with self-feedback,” *Advances in Neural Information Processing Systems*, vol. 36, pp. 46534–46594, 2023.
 - [16] Z. Zhou, J. Song, K. Yao, Z. Shu, and L. Ma, “ISR-LLM: Iterative self-refined large language model for long-horizon sequential task planning,” in *2024 IEEE International Conference on Robotics and Automation (ICRA)*, pp. 2081–2088, IEEE, 2024.
 - [17] S. Yao, D. Yu, J. Zhao, I. Shafraan, T. Griffiths, Y. Cao, and K. Narasimhan, “Tree of thoughts: Deliberate problem solving with large language models,” *Advances in Neural Information Processing Systems*, vol. 36, pp. 11809–11822, 2023.
 - [18] J. Wei, X. Wang, D. Schuurmans, M. Bosma, F. Xia, E. Chi, Q. V. Le, D. Zhou, *et al.*, “Chain-of-thought prompting elicits reasoning in large language models,” *Advances in Neural Information Processing Systems*, vol. 35, pp. 24824–24837, 2022.
 - [19] X. Wang, J. Wei, D. Schuurmans, Q. Le, E. Chi, S. Narang, A. Chowdhery, and D. Zhou, “Self-consistency improves chain of thought reasoning in language models,” *arXiv preprint arXiv:2203.11171*, 2022.
 - [20] Y. Kim, D. Kim, J. Choi, J. Park, N. Oh, and D. Park, “A survey on integration of large language models with intelligent robots,” *Intelligent Service Robotics*, vol. 17, no. 5, pp. 1091–1107, 2024.
 - [21] J. Wang, E. Shi, H. Hu, C. Ma, Y. Liu, X. Wang, Y. Yao, X. Liu, B. Ge, and S. Zhang, “Large language models for robotics: Opportunities, challenges, and perspectives,” *Journal of Automation and Intelligence*, vol. 4, no. 1, pp. 52–64, 2025.
 - [22] X. Huang, W. Liu, X. Chen, X. Wang, H. Wang, D. Lian, Y. Wang, R. Tang, and E. Chen, “Understanding the planning of LLM agents: A survey,” *CoRR*, 2024.
 - [23] L. Berti, F. Giorgi, and G. Kasneci, “Emergent abilities in large language models: A survey,” *arXiv preprint arXiv:2503.05788*, 2025.
 - [24] A. Billard, A. Albu-Schaeffer, M. Beetz, W. Burgard, P. Corke, M. Ciocarlie, R. Dahiya, D. Kragic, K. Goldberg, Y. Nagai, *et al.*, “A roadmap for AI in robotics,” *Nature Machine Intelligence*, vol. 7, no. 6, pp. 818–824, 2025.
 - [25] A. Brohan, N. Brown, J. Carbajal, Y. Chebotar, X. Chen, K. Choremanski, T. Ding, D. Driess, A. Dubey, C. Finn, *et al.*, “RT-2: Vision-language-action models transfer web knowledge to robotic control,” *arXiv preprint arXiv:2307.15818*, 2023.
 - [26] D. Driess, F. Xia, M. S. M. Sajjadi, C. Lynch, A. Chowdhery, B. Ichter, A. Wahid, J. Tompson, Q. Vuong, T. Yu, *et al.*, “PaLM-E: An embodied multimodal language model,” in *International Conference on Machine Learning*, PMLR, 2023.
 - [27] Gemini Robotics Team, “Gemini robotics 1.5: Pushing the frontier of generalist robots with advanced embodied reasoning, thinking, and motion transfer,” *arXiv preprint arXiv:2510.03342*, 2025.
 - [28] M. J. Kim, K. Pertsch, S. Karamcheti, T. Xiao, A. Balakrishna, S. Nair, R. Rafailov, E. P. Foster, P. R. Sanketi, Q. Vuong, *et al.*, “OpenVLA: An open-source vision-language-action model,” in *8th Annual Conference on Robot Learning*, 2024.
 - [29] W. Zhao, J. P. Queralta, and T. Westerlund, “Sim-to-real transfer in deep reinforcement learning for robotics: A survey,” in *2020 IEEE Symposium Series on Computational Intelligence (SSCI)*, pp. 737–744, IEEE, 2020.
 - [30] N. Koenig and A. Howard, “Design and use paradigms for Gazebo, an open-source multi-robot simulator,” in *2004 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS) (IEEE Cat. No. 04CH37566)*, vol. 3, pp. 2149–2154, IEEE, 2004.
 - [31] J. Collins, S. Chand, A. Vanderkop, and D. Howard, “A review of physics simulators for robotic applications,” *IEEE Access*, vol. 9, pp. 51416–51431, 2021.